

Feature Engineering for ML

A beginner's training guide - creating better features, taming skew, and reducing dimensions with scikit-learn

AI FOUNDATIONS - 2026

1 What Is Feature Engineering?

A machine-learning model is only as good as the information you feed it. **Feature engineering** is the craft of shaping that information - choosing, cleaning, transforming, and inventing the input columns (the X variables) so a model can learn the pattern more easily.

Think of raw data as raw ingredients and feature engineering as the prep work before cooking. The same ingredients, chopped and seasoned well, make a far better meal. Unlike plain scaling, feature engineering is tied to the specific problem you are solving and the thing you are trying to predict (the y variable).

[i] Key idea

Raw data contains both **signal** (the useful pattern) and **noise** (random clutter). Good features amplify the signal and hide the noise. A poorly designed feature will always produce a worse model than a well designed one - no algorithm can fully make up for it.

The three big activities

- **Feature creation** - build new, meaningful columns from existing ones (for example, adding three call-charge columns into one total).
- **Feature selection** - keep the useful columns and drop irrelevant or redundant ones.
- **Dimensionality reduction** - squeeze many columns into a few that still carry most of the information (this is where PCA comes in).

[!] Watch out

Two hazards to remember. **Overfitting**: too many or too-complex features let the model memorize the training data and fail on new data. **Data leakage**: information that wouldn't be available at prediction time sneaks into a feature, giving great scores in testing and terrible ones in the real world.

2 The Feature Engineering Workflow

Almost every project follows the same order of steps. Doing them in the right order - and fitting each step only on the training data - is what keeps leakage out.

Typical order

1. Load raw data (CSV -> DataFrame)
2. Clean it (fix types, handle missing values)
3. Split train / test <- BEFORE fitting anything
4. Encode categoricals (one-hot, etc.)
5. Scale / transform (min-max, log, power)
6. Create / reduce features (sums, ratios, PCA)
7. Train the model
8. Evaluate on the test set

Scikit-learn gives you two tools that keep this tidy. A `ColumnTransformer` applies different steps to different columns (say, one-hot encode the text columns while scaling the number columns). A `Pipeline` chains those steps together so they always run in the same order and only learn from the training data.

[+] Tip

A pipeline is your best defense against leakage. Because it fits every step on the training data and then applies the learned settings to the test data, you never accidentally let test information leak backward.

3 Turning Categories into Numbers

Most algorithms only understand numbers, but real data is full of labels like "Yes/No" or "International plan." **Encoding** converts those labels into numbers. The most common method is **one-hot encoding**: each category becomes its own 0/1 column, so no false ordering is implied.

churn example - preprocessing

```
from sklearn.preprocessing import OneHotEncoder, MinMaxScaler
from sklearn.compose import ColumnTransformer

categorical_columns = ['International plan', 'Voice mail plan']
numeric_columns = [c for c in X_train.columns if c not in categorical_columns]

# one-hot the text columns, min-max scale the numbers
cat = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
num = MinMaxScaler()

preprocessor = ColumnTransformer([
    ('cat', cat, categorical_columns),
    ('num', num, numeric_columns),
])
```

[i] Key idea

The setting `handle_unknown='ignore'` tells the encoder to quietly handle any category it never saw during training, instead of crashing. That matters when new values show up in real-world data.

4 Creating New Features

This is where **domain knowledge** pays off. If you understand the problem, you can combine raw columns into more meaningful ones. In the telecom churn example, day/evening/night charges were added into a single **total charge**, and minutes were divided by calls to get an **average call duration**.

feature creation

```
# combine related columns into totals
churn_df['total_charges'] = (churn_df['Total day charge']
                             + churn_df['Total eve charge']
                             + churn_df['Total night charge'])

# a ratio feature: minutes per call
churn_df['avg_day_call_duration'] = (
    churn_df['Total day minutes'] / churn_df['Total day calls'])
```

[!] Watch out

Ratio features can divide by zero, producing infinity. The fix used in the example: replace `inf` values with "missing," then fill those gaps with the column mean (**mean imputation**). Always check for nulls with `.isnull().sum()` after creating features.

You can also build a simple flag. In the house-price example, a **renovation indicator** was created: 1 if the home was ever renovated, 0 otherwise. And a **house age** feature came from subtracting the year built from the sale year - a small idea that gives the model something it couldn't easily infer on its own.

5 Which Features Actually Matter?

After training a **random forest** (a model made of many decision trees), you can ask it which features it leaned on most. These **feature importance** scores help you keep what helps and drop what doesn't.

feature importance

```
importances = pd.Series(  
    rf_model.feature_importances_, index=X_train.columns  
)  
.sort_values()  
importances.plot.barh(title='Random Forest Feature Importances')
```

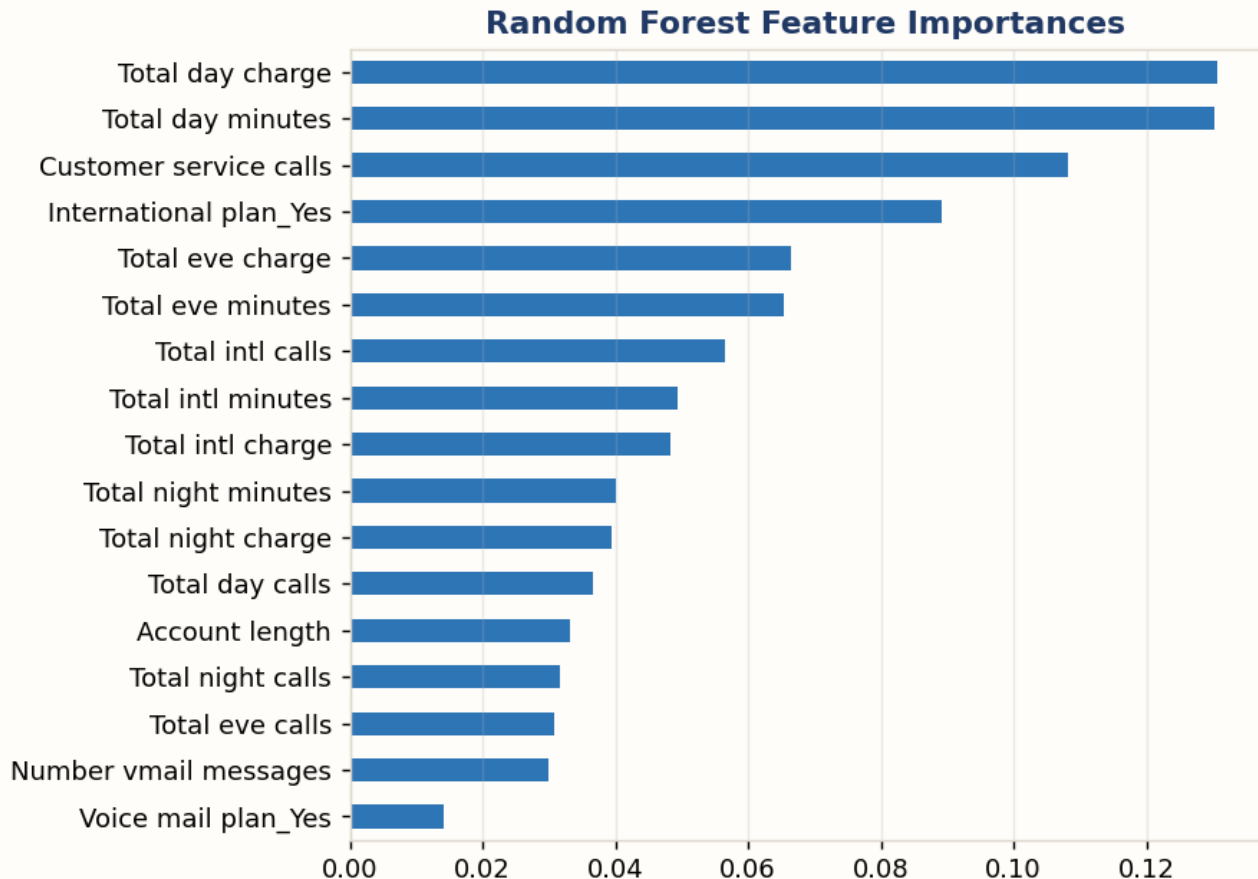


Figure 1. Feature importances from the telecom churn model. Longer bars are features the model relied on more. Charge and minute totals rise to the top - exactly the kind of engineered feature worth creating.

6 Polynomial Features for Curves

A plain **linear regression** draws a straight line. But many relationships curve. **Polynomial features** add powers of a column (x -squared, x -cubed) and interaction terms, letting a linear model bend to fit the data. The **degree** controls how much it can bend.

polynomial regression

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline

degree = 3
poly_model = make_pipeline(PolynomialFeatures(degree), LinearRegression())
poly_model.fit(X, y)
```

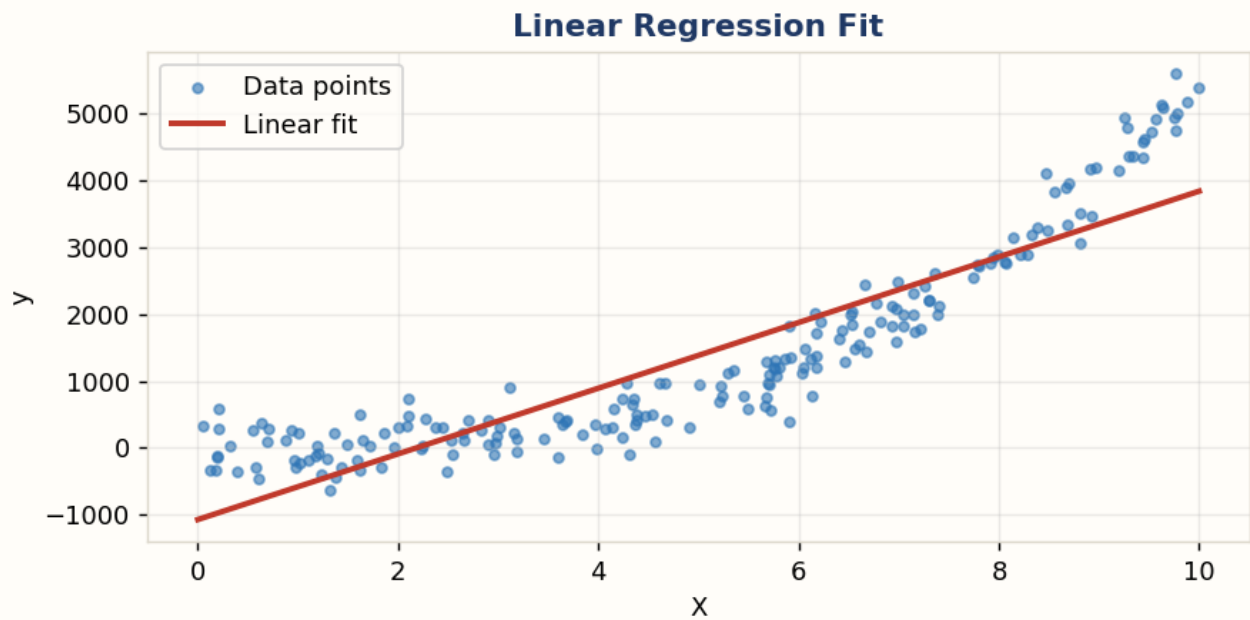


Figure 2. A straight line cannot follow this curved data - it underfits.

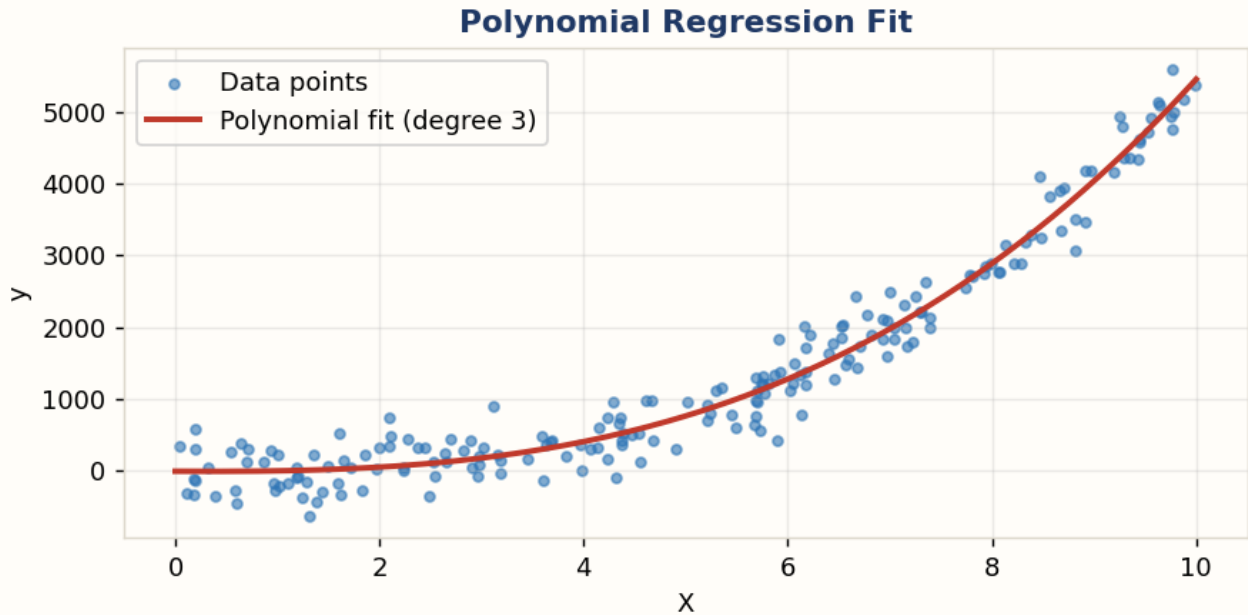


Figure 3. Adding degree-3 polynomial features lets the same linear model trace the curve closely.

[!] Watch out

Higher degree is not always better. Too high a degree and the curve wiggles through every point, memorizing noise - that's **overfitting**. Raising the degree also multiplies the number of features quickly, which adds computational cost.

7 Fixing Skewed Data

When a feature has a long tail - a few very large values dragging the distribution to the right - many models struggle. A **log transformation** compresses those large values and pulls the tail in, making the shape more symmetric (closer to the **normal** bell curve that models like).

log transform

```
from sklearn.preprocessing import FunctionTransformer
import numpy as np

# log1p computes log(1 + x) - safe even when x is 0
transform = FunctionTransformer(func=np.log1p)
X_log = transform.fit_transform(X)
```

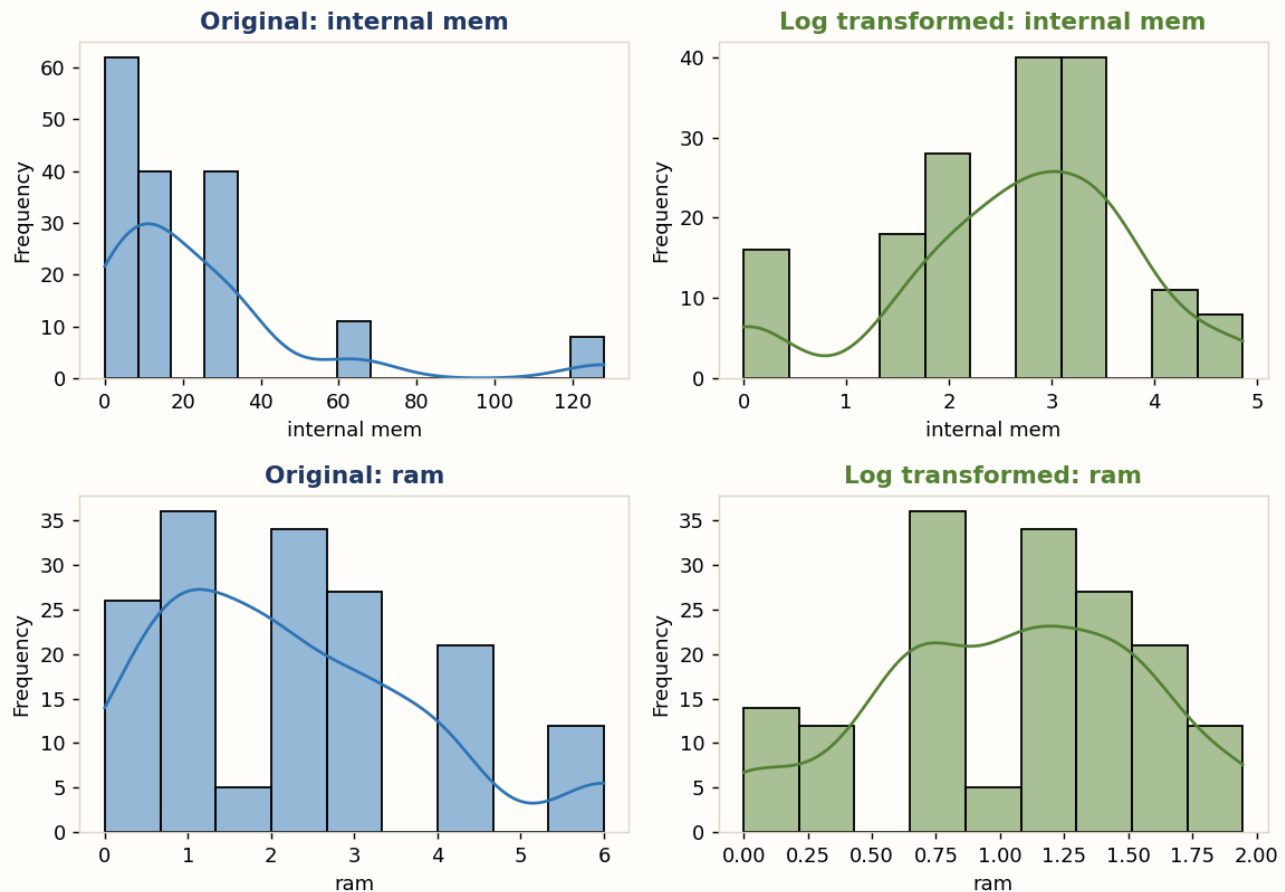


Figure 4. Before (blue) and after (green) a log transform. The lopsided left-heavy shapes become far more balanced, which helps the model.

[i] Key idea

Why `log1p` instead of plain `log`? Plain `log` of 0 is negative infinity, which breaks everything. `log1p` computes $\log(1 + x)$, so a zero value safely becomes $\log(1) = 0$.

For a more automatic option, a **power transformation** (`PowerTransformer`) applies the Yeo-Johnson method, which reshapes data toward normal and even handles zero and negative values. It's a good default when you're not sure which transform to use.

power transform

```
from sklearn.preprocessing import PowerTransformer
X_yeojohnson = PowerTransformer().fit_transform(X)
```

8 PCA: Fewer Columns, Same Story

When features overlap and correlate, you have more columns than you really need. **Principal Component Analysis (PCA)** compresses many correlated features into a few new ones - **principal components** - that capture most of the **variance** (the spread, i.e. the information). Each component is **uncorrelated** with the others.

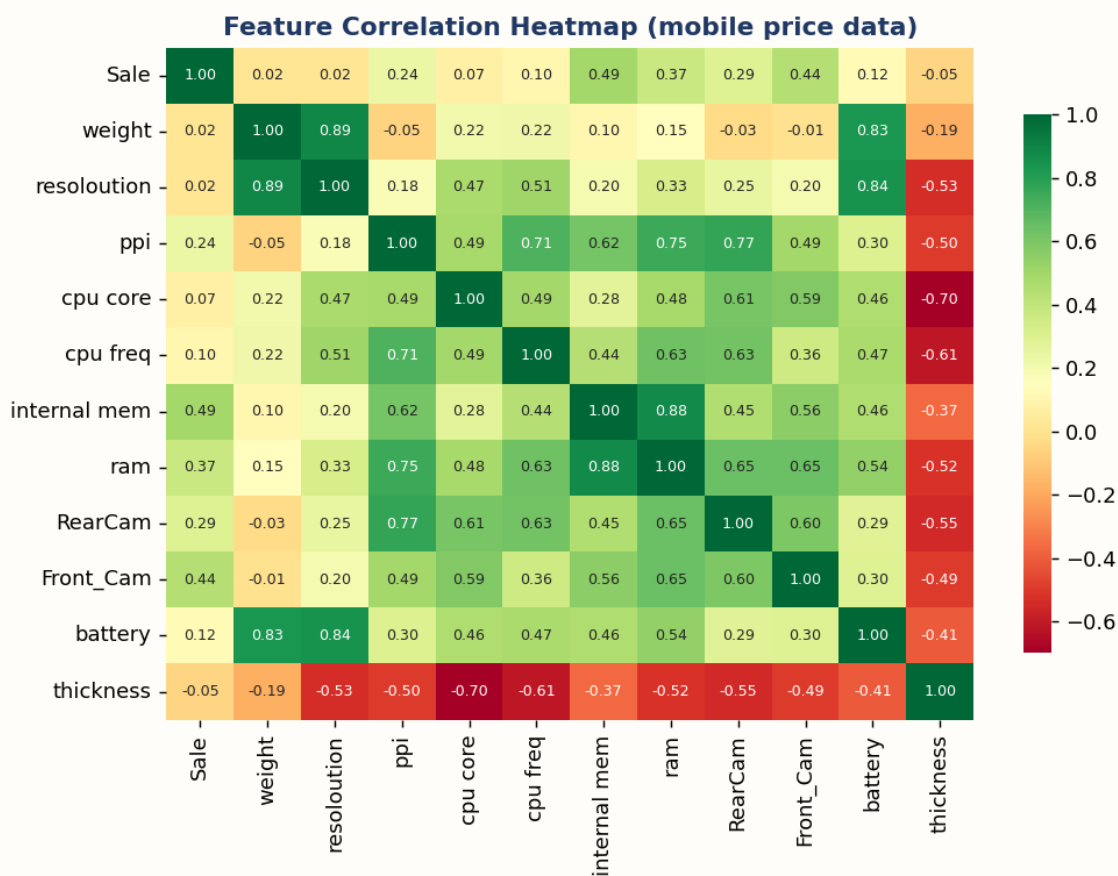


Figure 5. A correlation heatmap of the mobile-price features. Bright green and red squares show strongly correlated columns - a sign PCA can help.

How many components should you keep? Plot the **cumulative explained variance** and look for the **elbow** - the point where adding more components barely helps.

pca

```
from sklearn.decomposition import PCA

# keep 5 components
pca = PCA(n_components=5)
X_pca = pca.fit_transform(X_log)
print(pca.explained_variance_ratio_)
```

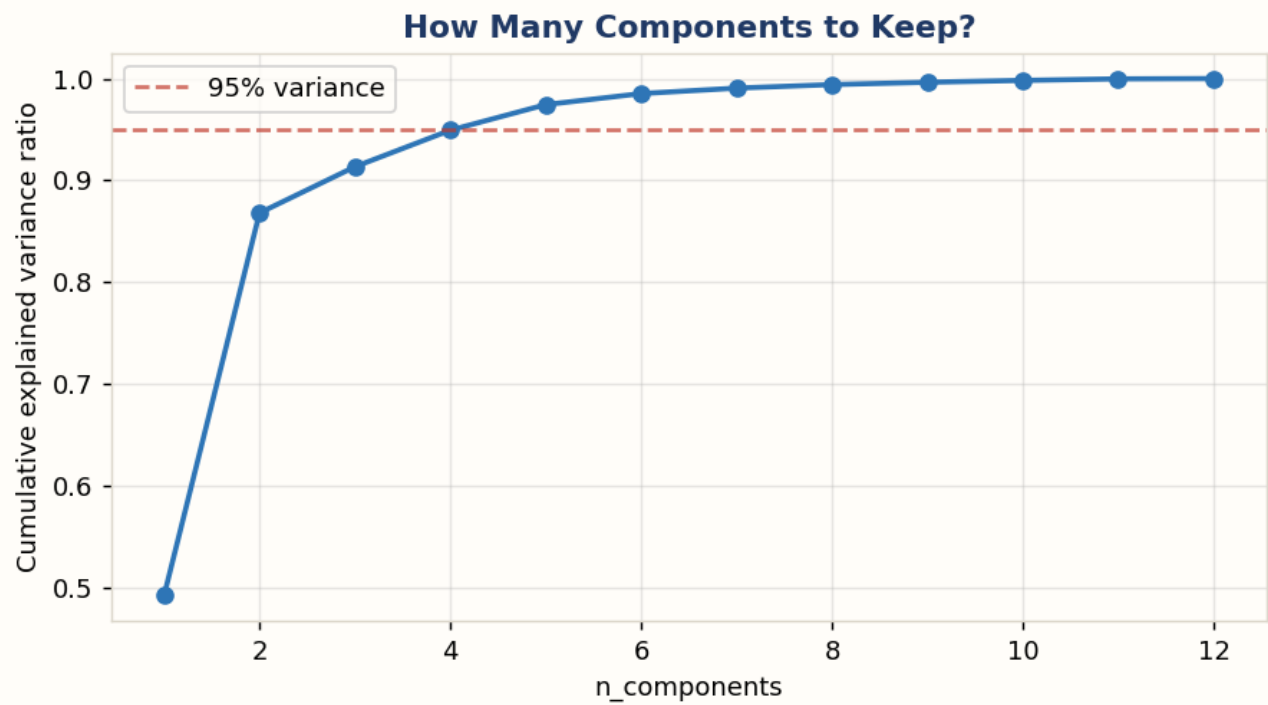



Figure 6. Cumulative explained variance. About 4-5 components already capture ~95% of the information, so the rest can be dropped.

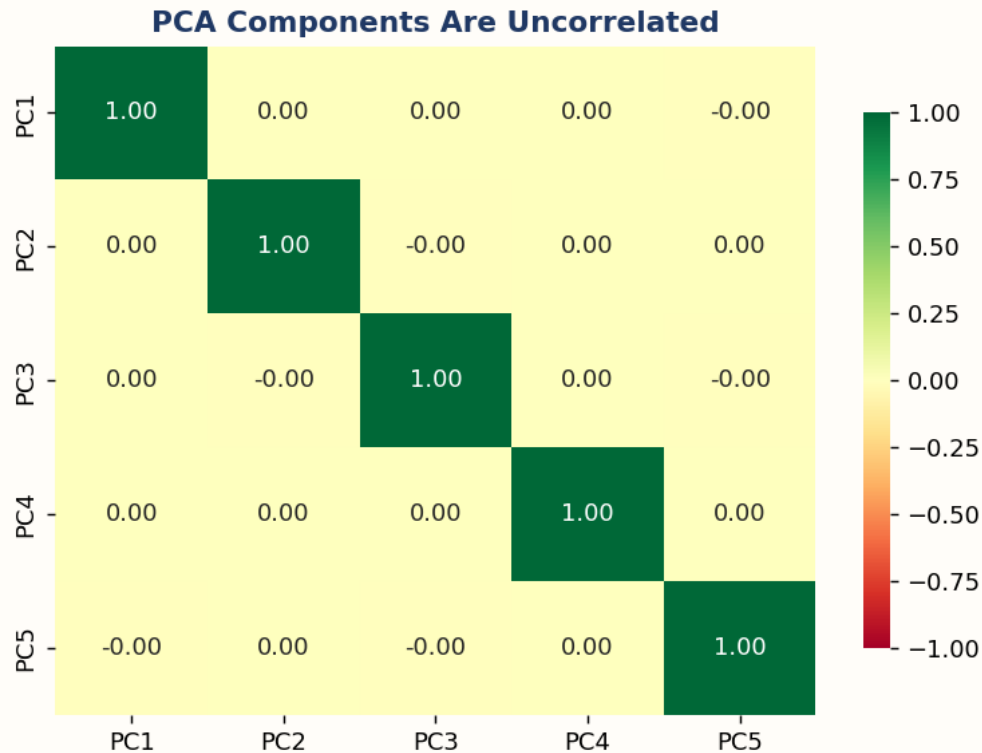


Figure 7. After PCA, the new components are uncorrelated - every off-diagonal value is essentially zero, exactly as intended.

⚠ Watch out

PCA's cost is **interpretability**. Each component is a blend of all the original features, so it no longer maps to a single real-world quantity. If you must explain predictions feature-by-feature, PCA may not be the right choice.

9 Vocabulary and Concepts

The key terms from this guide, in plain language. (A fuller alphabetical glossary is available as a companion document.)

Term	Plain-language meaning
Feature engineering	Shaping the input columns of a model to boost accuracy and reliability.
Feature (X variable)	An input column the model uses to make predictions.
Target (y variable)	The thing you are trying to predict.
Signal vs. noise	Signal is the useful pattern in data; noise is random clutter.

Term	Plain-language meaning
Overfitting	When a model memorizes the training data and fails on new data.
Generalization	How well a model performs on new, unseen data.
Data leakage	When information unavailable at prediction time slips into a feature, inflating scores.
Domain knowledge	Understanding of the subject area that guides which features to build.
Pipeline	A scikit-learn object chaining steps so they run in order and avoid leakage.
ColumnTransformer	Applies different preprocessing to different groups of columns.
Encoding	Converting category labels into numbers.
One-hot encoding	Making each category its own 0/1 column, implying no order.
Imputation	Filling in missing values (e.g. with the column mean).
Scaling	Rescaling numbers to a comparable range (min-max, standardization).
Feature importance	How much each feature contributed to a model's predictions.
Random forest	A model built from many decision trees; gives importance scores.
Polynomial features	Added powers and interactions that let a linear model fit curves.
Degree	How much a polynomial can bend; higher degree risks overfitting.
Skewed data	An asymmetric distribution with a long tail on one side.
Log transformation	Taking the log of values to compress large ones and reduce right skew.
log1p	$\log(1 + x)$ - a log transform that safely handles zeros.
Power transformation	Yeo-Johnson / Box-Cox reshaping of data toward a normal shape.
Normal distribution	The symmetric bell curve many methods prefer.
Variance	How spread out values are; PCA seeks to preserve it.
Dimensionality reduction	Cutting the number of features while keeping most information.
PCA	Compresses correlated features into a few uncorrelated components.
Principal components	The new uncorrelated axes PCA creates.
Explained variance ratio	The share of information each component captures.

Term	Plain-language meaning
Curse of dimensionality	The problems that arise when there are too many features.
Interpretability	How easily a human can understand why a model predicts what it does.

10 Further Reading for Students

These beginner-friendly resources go deeper on PCA and feature engineering, from interactive visual tools to step-by-step videos and official documentation.

Interactive & visual. PCA Explained Visually (Setosa.io) - drag data points and watch the components react.

<https://setosa.io/ev/principal-component-analysis/>

Video. StatQuest: PCA Step-by-Step - plain-English walkthrough, no heavy jargon.

<https://www.youtube.com/watch?v=FgakZw6K1QQ>

Video. StatQuest: PCA in Python - the theory turned into scikit-learn code with a scree plot.

<https://www.youtube.com/watch?v=Lsue2gEM9D0>

Docs & code. Scikit-Learn Official PCA User Guide - how PCA works and how to choose components.

<https://scikit-learn.org/stable/modules/decomposition.html#pca>

Tutorial. Real Python - normalizing data, fitting PCA, and choosing components by explained variance.

<https://realpython.com/>

Deep dive. Making Sense of PCA (Cross Validated) - intuition via the "2D photo of a 3D object" analogy.

<https://stats.stackexchange.com/questions/2691/making-sense-of-principal-component-analysis-eigenvectors-eigenvalues>

Deep dive. A Brain-Friendly Guide to PCA - bridges intuition and the eigenvector math with diagrams.

<https://markelic.de/a-brain-friendly-guide-to-pca/>

[+] Tip

New to all this? Start with the Setosa interactive tool to build visual intuition, then watch the two StatQuest videos, and finally try the scikit-learn code examples from this guide on your own dataset.

Feature Engineering for Machine Learning

A beginner's training guide

© 2026 David Grunwald. All rights reserved.